Smart PDF Notes Assistant: Grounded Study-Material Generation from Lecture PDFs with Lightweight Retrieval and a Local Language Model

Venkateshwaran M
Roll No. 240701584
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
Venkateshwaran.M.2024.cse@rajalakshmi.edu.in

Balaambiga C A
Roll No. 240701068
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
Balaambiga.CA.2024.cse@rajalakshmi.edu.in

***Abstract*—Students now study mostly from PDF lecture notes, and most of them already use general chatbots to summarise and explain them, yet those chatbots answer from their training data and can invent facts. This paper presents Smart PDF Notes Assistant, a Streamlit application that turns an uploaded PDF into a summary, likely examination questions, multiple-choice questions (MCQs) and a document-grounded chat, using retrieval-augmented generation (RAG). In its free local mode the system needs no API key: it indexes the PDF with TF-IDF and generates with a one-billion-parameter Llama 3.2 model served by Ollama; an OpenAI engine is optional. We evaluate the retrieval stage that grounds every output on 48 SQuAD articles treated as uploaded documents (10,570 questions). With 1,000-character chunks and four retrieved chunks, TF-IDF places the answer-bearing passage in the model's context for 91.0% of questions (BM25: 91.9%; random: 8.8%) while sending only 10.2% of the document. Recall falls to 65.6% for questions that share less than a third of their content words with the answer sentence. A 385-page PDF is ingested in 2.7 s on a two-core CPU, 88% of it in text extraction. A case study on computer-architecture notes shows a correct MCQ answer paired with an unfaithful explanation, which motivates answer verification.**

***Keywords*—retrieval-augmented generation, educational technology, question generation, TF-IDF, BM25, local language models, hallucination, PDF processing**

I. Introduction

Higher education in India enrolled about 4.33 crore students in 2021–22 [1], and much of their study material now arrives as PDF files: lecture slides exported to PDF, scanned or typeset notes and textbook chapters. These files can be long, and turning them into revision material is still manual work. At the same time, students have adopted generative AI quickly. A 16-country survey of 3,839 students found that 86% use AI in their studies, with searching for information and summarising documents among the most common uses [2], and a UK survey reported that use of any AI tool rose from 66% in 2024 to 92% in 2025, with explaining concepts and summarising articles as the leading tasks [3].

General chatbots are, however, poorly suited to studying from a specific document. They answer from their parameters rather than from the student's notes, so terminology and emphasis can drift from the syllabus, and they can state false content fluently. When asked for references, GPT-3.5 fabricated 55% of 222 citations and GPT-4 fabricated 18% of 414 [4], and half of the UK students surveyed said inaccurate results deter them from using AI [3]. Hallucination is a recognised, general weakness of neural text generation [5]. Learning science adds a second consideration: of ten common study techniques, practice testing was rated high utility while summarisation and rereading were rated low [6], and taking a test improves retention a week later more than restudying does [7]. A study assistant should therefore produce questions as well as summaries, and ground both in the student's own material.

Privacy and cost matter too. Notes may contain personal information that falls under India's Digital Personal Data Protection Act, 2023 [8], UNESCO's guidance on generative AI in education calls for data-protection safeguards [9], and paid API keys are a barrier for many students. Retrieval-augmented generation (RAG) [10] addresses grounding by retrieving relevant passages and conditioning the model on them, which reduces hallucination in knowledge-grounded dialogue [11]. This paper asks whether a deliberately lightweight RAG design, a sparse TF-IDF index and a small local model, can ground study material well enough on an ordinary student laptop. Its contributions are:

- **Smart PDF Notes Assistant**, a working application that generates a summary, likely examination questions with answers, MCQs and a document-grounded chat from one PDF, with a free local engine (TF-IDF retrieval and Llama 3.2 1B via Ollama) and an optional OpenAI engine.
- A reproducible evaluation of the retrieval stage on 10,570 questions over 48 documents, comparing the deployed TF-IDF retriever with BM25, keyword matching and random selection across four chunk sizes.
- Measurements of ingestion cost on PDFs of 9–385 pages, and of how much of a document is actually sent to the model.
- A failure analysis showing where lexical retrieval breaks down and a case study of an unfaithful MCQ explanation, with concrete design changes.

Section II reviews related work, Section III formulates the retrieval problem, Section IV describes the system, Section V gives the experimental setup, Section VI reports and discusses results, and Section VII concludes.

II. Related Work

A. Grounding language models in documents

RAG combines a retriever with a generator so that answers are conditioned on retrieved passages, which also lets the knowledge source change without retraining [10]; retrieval augmentation measurably reduces factual hallucination in dialogue [11]. A survey of RAG for large language models catalogues the design choices of chunking, retrieval and prompt construction that this work instantiates [12]. Grounding is not a cure-all: a benchmark of six models found that they struggle to refuse when the retrieved passages lack the answer and to integrate information spread across passages [13], and models use information in the middle of long inputs less reliably than at the start or end [14], an argument for sending a few relevant chunks rather than a whole document.

B. Lexical and dense retrieval

TF-IDF term weighting [15] and the probabilistic BM25 model [16] rank passages by shared, informative terms; they need no training and run in milliseconds on a CPU. Dense retrievers map questions and passages to learned vectors and outperform BM25 on open-domain question answering [17], and libraries such as FAISS make nearest-neighbour search over such vectors efficient [18]. Dense models, however, need an embedding model to be downloaded or called, which conflicts with a free, offline design; this work quantifies what the lexical choice costs.

C. Language models for learning

Kasneci et al. discuss opportunities and risks of large language models in education, including over-reliance and incorrect output [19]. A RAG chatbot grounded in a university data-science course answered 12,334 questions from 154 users in one semester, and focus-group participants judged its answers more accurate and course-specific than ChatGPT's [20]. A systematic review of automatic question generation found most systems produce simple factual questions and that plausible distractors remain hard [21]; in an educator study of LLM-generated MCQs, GPT-3.5 outperformed Llama 2 and Mistral on clarity and distractor selection, and smaller open models sometimes cited line positions instead of passages [22]. Text extraction itself is imperfect: rule-based PDF parsers degrade on scientific documents with formulas and graphics [23].

D. Positioning

Table I compares this work with the closest systems. To our knowledge, none of them combines document-grounded summaries, examination questions, MCQs and chat in one free, locally runnable tool, or reports how a lightweight retriever performs on the grounding step that all of those outputs depend on.

TABLE I
Comparison with Related Work

Work	Focus	Grounded in user's file	Runs locally, free
[10]	Open-domain QA	Yes (corpus)	No
[13]	RAG benchmark	Yes	–
[20]	Course chatbot	Yes (course)	No
[21]	Question generation	Varies	Varies
[22]	LLM MCQ generation	Paragraph in prompt	Partly
Ours	**Study tools from a PDF**	**Yes, retrieved**	**Yes**

III. Problem Formulation

A. Setting

A student uploads one PDF d. The system splits its text into chunks $c_1, \ldots, c_n$ and answers requests: a chat question, or a study tool (summary, important questions, MCQs) expressed as a task query. For each request it retrieves the k highest-scoring chunks, places them in a prompt and lets the language model generate. Everything the model writes about the document depends on whether the retrieved chunks contain the needed passage, so we evaluate this retrieval step directly.

B. Scoring functions

With term frequency $\mathrm{tf}(t, c)$, n chunks and document frequency $\mathrm{df}(t)$, the deployed retriever represents query and chunks with TF-IDF weights and ranks by cosine similarity:

$$w(t, c) = \mathrm{tf}(t, c) \cdot (\ln((1 + n) / (1 + \mathrm{df}(t))) + 1) \quad (1)$$

$$\mathrm{score}(q, c) = (\mathbf{w}_q \cdot \mathbf{w}_c) / (|\mathbf{w}_q|\,|\mathbf{w}_c|) \quad (2)$$

BM25 [16] instead saturates term frequency and normalises for chunk length $|c|$ against the mean length L:

$$K(c) = k_1 (1 - b + b\,|c| / L) \quad (3)$$

$$\mathrm{BM25}(q, c) = \Sigma_{t \in q}\, \mathrm{IDF}(t) \cdot \mathrm{tf}\,(k_1 + 1) / (\mathrm{tf} + K(c)) \quad (4)$$

with $k_1 = 1.5$ and $b = 0.75$. A keyword baseline counts distinct query terms present in a chunk, mimicking a reader's find function, and a random baseline orders chunks at random.

C. Measures

A question is a hit at k if one of the top-k chunks fully contains a gold answer span at its original position. Recall@k is the share of hits, and mean reciprocal rank MRR@10 averages 1/rank of the first hit (0 beyond rank 10). We also report the context sent to the model, the total length of the top-k chunks, against the length of the whole document.

IV. Proposed System

A. Architecture

Smart PDF Notes Assistant has five layers (Fig. 1). A Streamlit page holds the upload panel, the Process PDF and Clear File actions, and the engine settings. In the Free Local Engine no API key is needed: retrieval uses a TF-IDF index (shown in the interface as local-tfidf) and generation uses a model served by Ollama, by default Llama 3.2 1B, a model released for on-device use with a 128K-token context [24]. The OpenAI engine instead sends prompts to a hosted model with the user's key.

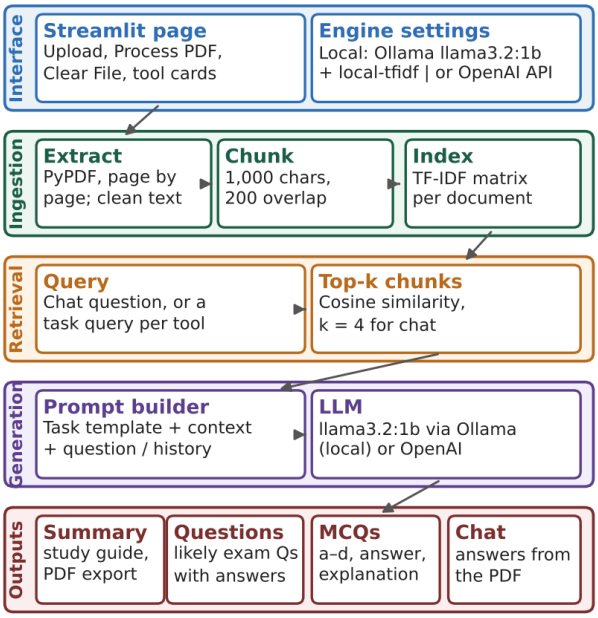


Fig. 1. Layered architecture of Smart PDF Notes Assistant. Ingestion runs once per document; retrieval and generation run for every request.

B. Ingestion

On Process PDF, text is extracted page by page with PyPDF, hyphenated line breaks are joined and stray line breaks inside sentences are removed. The text is split by a recursive character splitter that prefers paragraph, then line, then sentence boundaries, with chunks of at most 1,000 characters and 200 characters of overlap so that an idea crossing a boundary survives in at least one chunk. A TF-IDF vectoriser with English stop-word removal is fitted on the chunks of that document only, so its vocabulary and IDF weights reflect the student's notes.

C. Retrieval and generation

A chat question is used directly as the query with $k = 4$; each study tool uses a task query that describes what it needs, such as main topics, definitions and formulas. Algorithm 1 shows the request path. The prompt instructs the model to use only the supplied context and to say so when the context is insufficient. Outputs are kept in the session, marked Generated on their tool card (Fig. 2), and shown in a Generated Study Workspace with tabs for Chat with PDF, Summary, Important Questions and MCQs; the summary can be exported as a PDF study guide.

Algorithm 1: Handling one request

function Request(*index*, *model*, *r*)
  $q \leftarrow r$.question **if** *r* is chat **else** TaskQuery(*r*.tool)
  $C \leftarrow$ top-k chunks of *index* by score(q, $\cdot$) ▷ Eq. (2)
  ctx ← concatenate *C* in rank order
  p ← Template(*r*.tool) filled with *ctx*, *r* and recent chat turns
  a ← *model*.generate(*p*) ▷ Ollama or OpenAI
  store *a* in the session; mark the tool card Generated
  return *a*

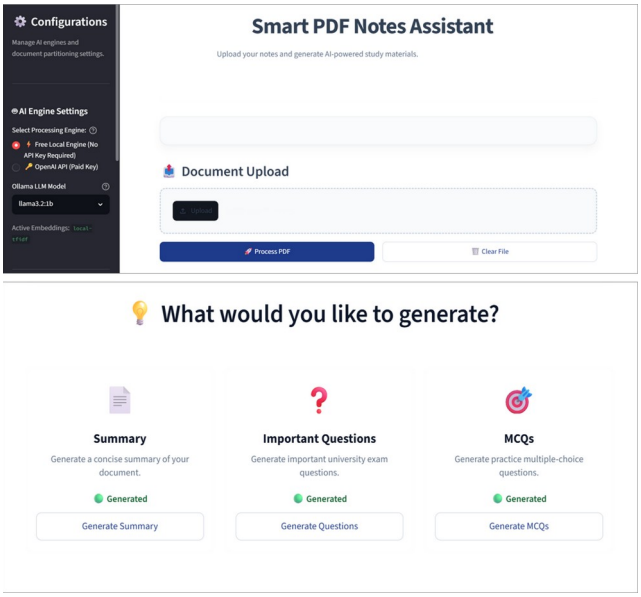


Fig. 2. The application. Top: engine selection (Free Local Engine with llama3.2:1b, embeddings local-tfidf) and document upload. Bottom: study tools, each marked Generated once produced.

V. Experimental Setup

A. Data

Student notes with answer annotations are not publicly available, so we use the SQuAD v1.1 development set [25] as a proxy: each of its 48 Wikipedia articles is treated as one uploaded document, and its 10,570 crowd-written questions, with answer spans marked in the text, serve as student queries. An article averages 33,344 characters, about nine A4 pages when typeset (Table II), close to a chapter of lecture notes. Because SQuAD questions were written while reading the passage, they share more vocabulary with it than students' questions may; Section VI-C examines this bias.

TABLE II
Evaluation Data

Item	Value
Documents (SQuAD v1.1 dev articles)	48
Questions with answer spans	10,570
Mean document length	33,344 chars (≈ 9 pages)
Chunks per document (1,000 / 200)	49.5 (mean 708 chars)

B. Procedure and environment

For each chunk size (250, 500, 1,000 and 2,000 characters, overlap one fifth of the size) every document is split and indexed separately, exactly as at upload, and every question is ranked against its own document's chunks. We report 95% bootstrap confidence intervals over questions (1,000 resamples). For ingestion cost, articles were typeset into A4 PDFs (11 pt Times) with ReportLab, and documents of 9 to 385 pages were built by concatenating articles; each stage was timed with the median of three runs. The pipeline used PyPDF, LangChain's text splitter, scikit-learn 1.9 [26] and rank-bm25 on a two-core Intel Xeon at 2.1 GHz under Linux, with no GPU.

VI. Results and Discussion

A. Retrieval accuracy

Table III and Fig. 3 give the results at the deployed setting of 1,000-character chunks. With four chunks, the TF-IDF retriever places an answer-bearing passage in the context for 91.0% of questions (95% CI 90.5–91.6%), against 8.8% for random selection, so the model almost always receives the evidence it needs. BM25 is slightly better, at 91.9% (91.4–92.5%), and its advantage is largest at the top rank: 77.5% against 72.8% at $k = 1$. Keyword matching, the analogue of a reader's find function, trails at 89.5% and 71.9%. Accuracy varies by document: TF-IDF recall@4 ranges from 77.4% to 100% across the 48 articles, with a median of 93.2%.

TABLE III
Retrieval with 1,000-Character Chunks (%)

Method	R@1	R@4	R@8	MRR@10
Random	2.2	8.8	17.7	6.4
Keyword match	71.9	89.5	94.2	80.2
TF-IDF (deployed)	72.8	91.0	94.9	81.4
BM25	77.5	91.9	95.5	84.4

R@k: recall at k over 10,570 questions; MRR@10 × 100. 95% CI for R@4: TF-IDF 90.5–91.6, BM25 91.4–92.5.

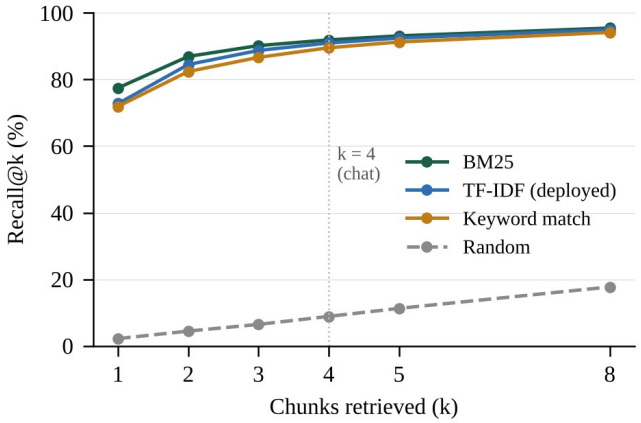


Fig. 3. Recall@k for each retriever at 1,000-character chunks. All lexical methods exceed 89% by k = 4; BM25 leads at small k.

B. Chunk size and context

Larger chunks raise recall but cost context (Table IV, Fig. 4). Moving from 250- to 1,000-character chunks lifts TF-IDF recall@4 from 83.0% to 91.0%; doubling again to 2,000 characters adds only 2.0 points while sending 2.2 times as much text. At the deployed setting the four chunks average 2,886 characters, 10.2% of the document, or roughly 700 tokens. For a one-billion-parameter model, which must find the relevant sentence inside its prompt [14], we regard 1,000 characters as the better balance.

TABLE IV
Effect of Chunk Size (TF-IDF, k = 4)

Chunk size	Chunks/doc	Context (≈tokens)	R@4 (%)
250	177.0	223	83.0
500	96.8	397	88.4
1,000 (deployed)	49.5	708	91.0
2,000	21.8	1,551	93.0

Context is four times the mean chunk length, at about four characters per token.

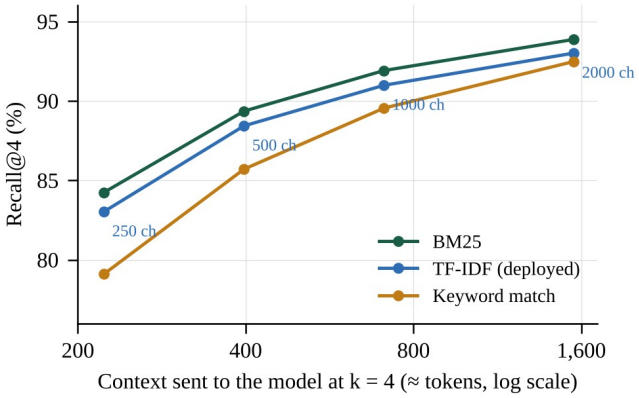


Fig. 4. Recall@4 against the context sent to the model for chunk sizes of 250–2,000 characters. Gains flatten beyond 1,000 characters.

C. Where lexical retrieval fails

We grouped questions by the share of their content words that also occur in the sentence containing the answer (Table V). When the overlap is high, every lexical method finds the passage for 98.7–99.5% of questions; when it is below one third, TF-IDF recall@4 falls to 65.6%. These 1,609 questions, 15.2% of the set, are paraphrases, the way a student who has not yet learned the vocabulary of a topic may ask. Since SQuAD favours high overlap, real student questions are likely to fall in this weaker band more often, so this is the main limitation of the free engine. Dense retrieval is designed for exactly this case [17], and a hybrid that adds dense scores to BM25 is the natural next step where an embedding model can be run locally.

TABLE V
Recall@4 by Question–Answer Lexical Overlap (%)

Overlap	Questions	Keyword	TF-IDF	BM25
Low (< 1/3)	1,609	57.9	65.6	66.2
Medium	5,235	92.2	93.3	94.6
High (> 2/3)	3,726	99.5	98.7	99.3

D. Ingestion cost

Ingestion scales linearly and stays interactive (Fig. 5): a 9-page document is ready in 0.06 s and a 385-page document in 2.72 s, of which 2.38 s, 88%, is PyPDF text extraction. Building the TF-IDF index for 2,412 chunks takes 0.15 s, and a query is scored in 1.6 ms. Retrieval therefore adds nothing noticeable to a request; the time a student waits is spent in the language model. Extracted text matched the source with a similarity ratio above 0.999, but these PDFs were generated cleanly; scanned notes and pages dense with formulas extract far worse [23] and would need OCR.

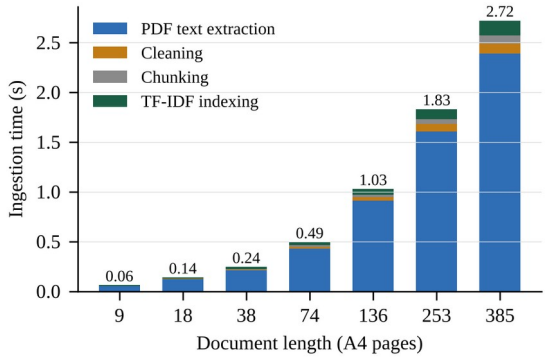


Fig. 5. Ingestion time by stage for PDFs of 9–385 pages on a two-core CPU. Text extraction dominates; indexing takes at most 0.15 s.

E. Case study: computer-architecture notes

We ran the application on a student's notes on pipeline hazards with the local engine. The exported summary was organised into an executive summary, key concepts, definitions and formulas drawn from the notes, and the important questions came with answers that distinguished structural, data and control hazards. The MCQ in Fig. 6 asks for the primary cause of data hazards and marks read-after-write correctly, but its explanation attributes data hazards to insufficient hardware resources, which is the definition of a structural hazard; one distractor, stalling, is a remedy rather than a cause. Such an error is easy for a learner to absorb, because the answer key is right. It reflects the limits of a one-billion-parameter model and the integration weakness reported for RAG systems [13], and it argues for a verification step that checks each explanation against the retrieved passage before display, and for flagging generated items as drafts to be checked.

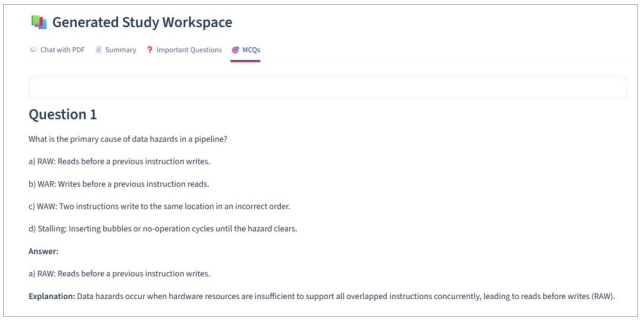


Fig. 6. An MCQ generated by the local engine from notes on pipeline hazards. The answer is correct, but the explanation describes a structural hazard.

F. Limitations and threats to validity

The retrieval study uses Wikipedia articles and crowd-written questions as a stand-in for notes and student queries, and measures whether evidence reaches the model, not whether the final text is correct; recall bounds output quality but does not establish it. Generation was examined qualitatively on one set of notes, without a learning-outcome study. Timings come from one machine, and the PDFs were born-digital. The evaluation scripts are released with this paper so that each figure can be reproduced on a student laptop.

VII. Conclusion and Future Work

Smart PDF Notes Assistant shows that a deliberately lightweight retrieval-augmented design can ground study material on ordinary hardware without paid services. A per-document TF-IDF index puts the answer-bearing passage in the model's context for 91.0% of 10,570 questions while sending about a tenth of the document, ingests a 385-page PDF in under three seconds, and lets a one-billion-parameter local model produce summaries, examination questions, MCQs and grounded chat. The evaluation also identifies what to fix: retrieval drops to 65.6% for paraphrased questions, BM25 is a free improvement at the top rank, and a small model can pair a correct answer with an unfaithful explanation.

Future work will replace TF-IDF with BM25 and then a hybrid lexical–dense retriever, add a verification pass that checks generated explanations and MCQ keys against the retrieved text, add OCR for scanned notes, support flashcards and spaced repetition, and evaluate learning outcomes with students on real course notes.

References

[1] Ministry of Education, Government of India, "All India Survey on Higher Education 2021–22," New Delhi, India, 2024. [Online]. Available: https://education.gov.in/

[2] Digital Education Council, "Global AI Student Survey 2024," Aug. 2024. [Online]. Available: https://www.digitaleducationcouncil.com/

[3] J. Freeman, "Student Generative AI Survey 2025," Higher Education Policy Institute and Kortext, London, U.K., Feb. 2025. [Online]. Available: https://www.hepi.ac.uk/

[4] W. H. Walters and E. I. Wilder, "Fabrication and errors in the bibliographic citations generated by ChatGPT," *Sci. Rep.*, vol. 13, Art. no. 14045, 2023.

[5] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, Art. no. 248, 2023.

[6] J. Dunlosky, K. A. Rawson, E. J. Marsh, M. J. Nathan and D. T. Willingham, "Improving students' learning with effective learning techniques: Promising directions from cognitive and educational psychology," *Psychol. Sci. Public Interest*, vol. 14, no. 1, pp. 4–58, 2013.

[7] H. L. Roediger and J. D. Karpicke, "Test-enhanced learning: Taking memory tests improves long-term retention," *Psychol. Sci.*, vol. 17, no. 3, pp. 249–255, 2006.

[8] Government of India, "The Digital Personal Data Protection Act, 2023 (No. 22 of 2023)," *Gazette of India*, Aug. 2023.

[9] F. Miao and W. Holmes, *Guidance for Generative AI in Education and Research*. Paris, France: UNESCO, 2023.

[10] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 9459–9474.

[11] K. Shuster, S. Poff, M. Chen, D. Kiela and J. Weston, "Retrieval augmentation reduces hallucination in conversation," in *Findings Assoc. Comput. Linguist.: EMNLP 2021*, 2021, pp. 3784–3803.

[12] Y. Gao *et al.*, "Retrieval-augmented generation for large language models: A survey," arXiv:2312.10997, 2023.

[13] J. Chen, H. Lin, X. Han and L. Sun, "Benchmarking large language models in retrieval-augmented generation," in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 16, 2024, pp. 17754–17762.

[14] N. F. Liu *et al.*, "Lost in the middle: How language models use long contexts," *Trans. Assoc. Comput. Linguist.*, vol. 12, pp. 157–173, 2024.

[15] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manage.*, vol. 24, no. 5, pp. 513–523, 1988.

[16] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009.

[17] V. Karpukhin *et al.*, "Dense passage retrieval for open-domain question answering," in *Proc. EMNLP*, 2020, pp. 6769–6781.

[18] J. Johnson, M. Douze and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[19] E. Kasneci *et al.*, "ChatGPT for good? On opportunities and challenges of large language models for education," *Learn. Individ. Differ.*, vol. 103, Art. no. 102274, 2023.

[20] M. Thway, J. Recatala-Gomez, F. S. Lim, K. Hippalgaonkar and L. W. T. Ng, "Battling botpoop using GenAI for higher education: A study of a retrieval augmented generation chatbot's impact on learning," arXiv:2406.07796, 2024.

[21] G. Kurdi, J. Leo, B. Parsia, U. Sattler and S. Al-Emari, "A systematic review of automatic question generation for educational purposes," *Int. J. Artif. Intell. Educ.*, vol. 30, pp. 121–204, 2020.

[22] G. Biancini, A. Ferrato and C. Limongelli, "Multiple-choice question generation using large language models: Methodology and educator

insights," in *Adjunct Proc. 32nd ACM Conf. User Modeling, Adaptation and Personalization (UMAP Adjunct '24)*, 2024.

[23] N. S. Adhikari and S. Agarwal, "A comparative study of PDF parsing tools across diverse document categories," arXiv:2410.09871, 2024.

[24] Hugging Face, "Llama can now see and run on your device – welcome Llama 3.2," Sep. 25, 2024. [Online]. Available: https://huggingface.co/blog/llama32

[25] P. Rajpurkar, J. Zhang, K. Lopyrev and P. Liang, "SQuAD: 100,000+ questions for machine comprehension of text," in *Proc. EMNLP*, 2016, pp. 2383–2392.

[26] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.